SYSTEM DESIGN · CAPSTONE BLUEPRINT

AgentForge

The AI Software Company

Complete System Design and Seven-Day Capstone Blueprint

A multi-agent platform that turns an initial product idea into a working application, then maintains it over time by resolving bug reports and feature tickets through coordinated AI agents, persistent project memory, automated verification, and human-controlled releases.

CORE IDEA

Build an AI engineering organization, not a chatbot. The user describes a product. AgentForge plans and builds it. Later, the user submits tickets, and the same system investigates, changes, tests, reviews, and prepares those changes for release while preserving the knowledge needed to maintain the project.

Design principle: agents propose and execute bounded work; the platform owns state, permissions, budgets, verification, and release policy.

Contents

1. Executive Summary . . . 3
2. Requirements and System Boundaries . . . 4
3. High-Level Architecture . . . 5
4. End-to-End Workflows . . . 7
5. Agent Roles and Runtime . . . 8
6. Project Brain: Persistent Memory . . . 10
7. Data Model and Persistence . . . 12
8. API and Event Design . . . 13
9. Orchestration, Reliability, and Concurrency . . . 14
10. Security and Trust Boundaries . . . 16
11. Observability, Cost, and Evaluation . . . 17
12. Frontend and Deployment Design . . . 18
13. Seven-Day Implementation Plan . . . 19
14. Scope, Risks, and Final Demonstration . . . 20
15. Architecture Decisions and Glossary . . . 21

What this document covers

- Product scope, requirements, and quality attributes
- High-level architecture and end-to-end workflows
- Agent roles, orchestration, task state, and coordination
- Project Brain: memory, retrieval, provenance, and freshness
- Database model, APIs, event design, and repository execution
- Security, reliability, observability, and evaluation
- Seven-day build plan and final demo criteria

DESIGN PRINCIPLE

Agents propose and execute bounded work; the platform owns state, permissions, budgets, verification, and release policy.

SECTION 01

Executive Summary

AgentForge is a persistent, multi-agent software engineering platform. It starts with a user's idea, creates a plan and repository, implements a first working version, and then continues evolving that codebase through tickets. A ticket may describe a bug, a feature, a refactor, a security issue, or a performance problem.

The platform coordinates specialized agents such as Product Manager, Architect, Engineering, QA, Security, Reviewer, and Release Manager. These agents operate through a controlled runtime with structured inputs and outputs, permissioned tools, task dependencies, execution budgets, and verifiable artifacts.

> **THE CENTRAL ARCHITECTURAL DECISION**
> Separate agent intelligence from system control. Agents reason, plan, propose patches, and analyze failures. Deterministic services own the workflow state machine, task scheduling, authorization, retry policy, memory provenance, verification requirements, and approval gates.

1.1 Product lifecycle

Stage	What AgentForge does	Evidence produced
Idea intake	Clarifies goals, constraints, users, and desired outcomes.	Requirements and acceptance criteria
Architecture	Proposes components, interfaces, data model, and technical decisions.	Architecture documents and ADRs
Planning	Breaks work into dependent, verifiable tasks.	Task DAG and execution plan
Initial build	Runs coding agents in controlled workspaces and integrates changes.	Commits, patches, test reports
Ticket maintenance	Reconstructs context, investigates bugs, and implements requested changes.	Ticket-linked artifacts and regression tests
Verification	Runs tests, quality checks, and security review.	Trusted check results
Release	Prepares a pull request or release candidate for authorized approval.	Reviewable diff and release record
Learning loop	Records verified outcomes and refreshes project knowledge.	Versioned memory records

1.2 Explicit non-goals for the first version

- Do not support every programming language, Git provider, or deployment target.
- Do not let agents merge or deploy arbitrary code without authorization.
- Do not treat chat history or vector search as the authoritative source of truth.
- Do not mark a ticket complete because an agent claims it is done.
- Do not begin with many microservices; use a modular monolith plus isolated execution workers.

SECTION 02

Requirements and System Boundaries

2.1 Functional requirements

- **Project creation:** accept an idea, constraints, supported stack, and target outcomes.
- **Requirements generation:** produce user stories, acceptance criteria, assumptions, and open questions.
- **Planning:** create tasks with dependencies, ownership, expected artifacts, and verification criteria.
- **Repository integration:** connect or initialize a repository, inspect code, and create isolated workspaces.
- **Agent execution:** invoke models, tools, and role-specific agents under explicit budgets.
- **Ticket lifecycle:** support bug reports, features, refactors, blocked work, cancellations, and review feedback.
- **Project Brain:** retrieve relevant source code, decisions, historical runs, tests, and product requirements.
- **Verification:** run actual tests and checks in trusted workers and attach results to a specific code revision.
- **Review and release:** produce diffs and pull requests; require authorization for merge and deployment.
- **Observability:** show current run state, task graph, agent actions, failures, latency, and costs.

2.2 Quality attributes

Attribute	Design requirement
Durability	Persist state transitions and artifacts; resume interrupted work safely.
Correctness	Verify code at a known revision; do not trust model assertions as evidence.
Isolation	Scope project data, repository access, credentials, and workers to authorized tenants.
Reliability	Assume at-least-once job delivery; make operations idempotent where possible.
Cost control	Set per-run token, wall-clock, tool-call, retry, and concurrency budgets.
Traceability	Link ticket → task → agent run → tool calls → artifacts → commit → checks.
Maintainability	Use explicit module boundaries, schemas, migrations, and versioned contracts.
Security	Treat prompts, repository files, tool output, and generated code as untrusted.
Recoverability	Use checkpoints, durable events, and controlled retry/resume semantics.

2.3 Core domain objects

Organization/User, Project, Repository, Ticket, Task, TaskDependency, AgentDefinition, AgentRun, ToolCall, Artifact, TaskEvent, MemoryRecord, ArchitectureDecision, PullRequest, CheckResult, Approval, UsageRecord, and Release.

SECTION 03

High-Level Architecture

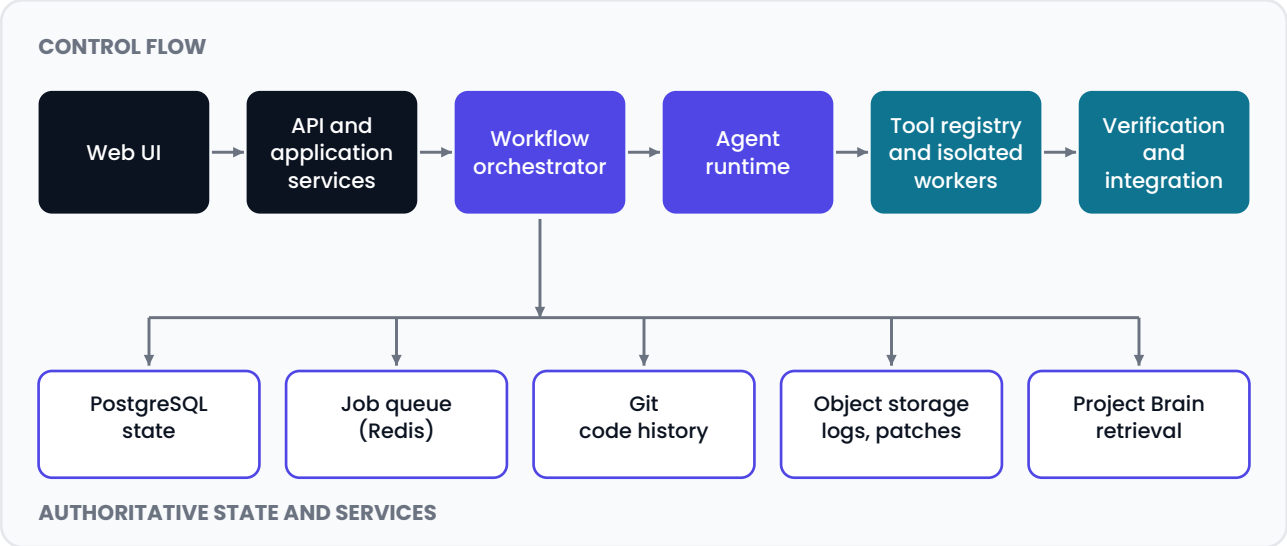
RECOMMENDED STARTING POINT

A modular monolith for the product/API and orchestration control plane, backed by PostgreSQL and a job queue, with separate isolated workers for code execution. This keeps the first implementation understandable while preserving the security boundary around generated code.

3.1 Logical components

Component	Responsibilities
Web application	Project setup, ticket backlog, run timeline, agent activity, code diffs, memory explorer, approvals.
API/application layer	Authentication, project and ticket APIs, repository integration, authorization, request validation.
Workflow orchestrator	Durable state machine, task DAG, scheduling, retries, budgets, cancellation, approval gates.
Agent runtime	Model adapter, role instructions, context assembly, structured outputs, tool registry, policy checks.
Project Brain	Code index, architecture decisions, product knowledge, execution history, retrieval, freshness and provenance.
Job queue/workers	Asynchronous dispatch, long-running tasks, concurrency limits, retries, sandboxed execution.
Verification/integration	Test execution, lint/build checks, security checks, patch integration, pull request preparation.
Persistence/storage	PostgreSQL for state, Git for code history, object storage for large artifacts, optional vector search.
Observability	Structured logs, metrics, traces, event history, token usage, run-level cost and quality metrics.

3.2 Component diagram



Web UI → API and application services → workflow orchestrator → agent runtime → tool registry and isolated workers → verification and integration. PostgreSQL stores authoritative state; a queue dispatches work; Git stores code and history; object storage stores large logs, patches, and reports. Project Brain retrieves relevant context from these sources.

3.3 Why not microservices first?

Agent orchestration, retries, model calls, and sandbox execution already introduce complexity. A modular monolith reduces deployment and debugging overhead. Keep the execution worker boundary separate because it runs untrusted code. Extract services later only when scale, isolation, or team ownership justifies it.

3.4 Recommended stack

Layer	Suggested technology
Frontend	Next.js, React, TypeScript, Tailwind CSS, component library
Backend	Node.js + TypeScript; NestJS or another modular server framework
Database	PostgreSQL; Prisma or Drizzle for schema and migrations
Queue	Redis + BullMQ for the initial job system; add durable workflow tooling if needed
Agent runtime	Provider-neutral LLM adapter, structured outputs, tool calling, role configurations
Code execution	Git worktrees and isolated Docker workers with restricted resources and networking
Search/memory	PostgreSQL full-text search; pgvector for semantic retrieval when useful
Testing	Vitest/Jest, integration tests, Playwright end-to-end tests
CI/CD	GitHub Actions or equivalent provider pipeline
Observability	OpenTelemetry, structured logs, metrics, and run traces

SECTION 04

End-to-End Workflows

4.1 Initial application build

1. User submits an idea and constraints.
2. Requirements Agent creates requirements, acceptance criteria, assumptions, and open questions.
3. Architect Agent proposes modules, interfaces, data model, and architecture decisions.
4. Planning Agent creates a task DAG and defines verification requirements.
5. Orchestrator validates dependencies, budgets, and permitted tools.
6. Coding agents implement bounded tasks in isolated workspaces.
7. Integration combines compatible patches and resolves conflicts through explicit review.
8. QA and Security agents request checks; trusted workers execute them.
9. Failed checks create bounded repair tasks; repeated failures trigger escalation.
10. A release candidate is prepared for human approval.
11. Verified decisions and outcomes are recorded in Project Brain.

4.2 Ticket maintenance flow

1. Create and classify a ticket.
2. Retrieve relevant project memory and inspect the current repository revision.
3. Assemble a context packet with source references, constraints, and unresolved hypotheses.
4. Reproduce the bug or validate the requested behavior when possible.
5. Plan a minimal implementation and identify impacted modules and tests.
6. Implement the change in an isolated workspace.
7. Run targeted tests, then regression and security checks as required.
8. Review the diff and request bounded repairs if necessary.
9. Prepare a pull request with evidence and traceable artifacts.
10. Require authorized approval before merge or deployment.
11. Update memory with verified root cause, code revision, tests, and decisions.

4.3 Ticket state machine

State	Meaning / allowed transition
Backlog	Ticket exists but work has not begun; may move to Planning or Cancelled.
Planning	Requirements and dependencies are being prepared; then Ready or Blocked.
Ready	Required inputs and dependencies are satisfied; scheduler may dispatch work.
In Progress	One or more task runs are active.
Verifying	Tests and policy checks are running against a known revision.
Review	A reviewable change exists; reviewer may approve or request changes.
Blocked	Missing information, permissions, budget, or dependency prevents safe progress.
Approved	Required human/policy approvals are recorded.
Merged / Released	Integration or deployment has been confirmed by authoritative evidence.
Cancelled	Work is stopped with a recorded reason and safe cleanup.

4.4 Repair loop

BOUNDED BY DESIGN

A failed check creates a structured failure artifact with the command, exit code, test name, relevant output, and code revision. The agent receives that evidence and may propose a patch. Retries are bounded by count, time, and cost. If the same failure persists, the workflow pauses for human intervention instead of looping forever.

SECTION 05

Agent Roles and Runtime

Agents are configured execution roles, not necessarily separate models or independent services. Each role defines purpose, allowed tools, input/output schemas, context requirements, permission scope, and budgets.

Role	Primary responsibility	Output artifact
Requirements Agent	Turn product intent into testable requirements.	Requirements document, acceptance criteria
Architect Agent	Design components, interfaces, schema, trade-offs.	Architecture document, ADR proposals
Planning Agent	Create task dependencies and sequencing.	Task DAG with owners and verification criteria
Engineering Agent	Inspect code and implement bounded changes.	Patch, implementation notes, test requests

Role	Primary responsibility	Output artifact
QA Agent	Reproduce behavior and verify acceptance criteria.	Test report and reproducible failures
Security Agent	Review code and architecture against threat model.	Findings, severity, evidence, remediation
Review Agent	Assess correctness, maintainability, and scope.	Approval or requested changes
Release Agent	Prepare release candidate and release notes.	Release checklist and deployment proposal

5.1 Agent invocation contract

Every invocation should receive a structured envelope containing `project_id`, `ticket_id`, `task_id`, `run_id`, `role`, `base_commit`, task description, acceptance criteria, context references, allowed tools, permission scope, deadline, and budgets. Outputs should include artifacts, files changed, tests requested, observed results, assumptions, and unresolved issues.

5.2 Tool registry

- **Repository tools:** read file, search symbols, inspect diff, apply patch, create branch/worktree.
- **Execution tools:** run allowlisted commands, run tests, lint, build, inspect logs.
- **Project tools:** read ticket, retrieve approved memory, propose ADR, attach artifact.
- **Workflow tools:** request task, report progress, request clarification, request human approval.
- Every tool call is validated outside the model and logged with input/output references, status, duration, and permission decision.

5.3 Parallelism and shared files

Parallelize independent tasks only after interfaces are stable. Use isolated Git worktrees, branch/patch boundaries, file ownership hints, and integration checks. Detect overlapping edits before integration. Conflicts should become explicit work items or be resolved by a designated integration step, not silently overwritten.

5.4 Model routing

Start with one model provider and role-specific instructions. Introduce model routing only after benchmarks show that a model change improves quality, latency, or cost. Track actual token usage and estimated cost per agent run and per ticket.

SECTION 06

Project Brain: Persistent Memory

WHAT PROJECT BRAIN IS

A context retrieval and knowledge-management subsystem over authoritative project data. It should not be a single giant prompt or a vector database that is treated as truth.

Memory layer	Authoritative source / storage	Use
Codebase memory	Git repository and commit history	Current implementation, files, tests, dependency changes
Architectural memory	Versioned ADRs and design documents	Why decisions were made; constraints and rejected options
Episodic memory	Run records, events, artifacts	Previous attempts, failures, fixes, and outcomes
Semantic knowledge	Search index and optional embeddings	Find related incidents, concepts, and decisions by meaning
Operational memory	Timestamped structured records	Known issues, flaky tests, current warnings, release status
Product memory	Requirements, tickets, acceptance criteria	Intended behavior, priorities, and business rules

6.1 Context assembly on a new ticket

The Context Builder combines exact search (ticket IDs, file paths, symbols), repository analysis at the current commit, structured queries (tests, status, ownership), and semantic retrieval (similar bugs, decisions, incidents). It ranks results by relevance, authority, freshness, and access permissions, then produces a task-specific context packet.

6.2 Example context packet

Field	Content
Ticket	Duplicate checkout charges.
Current code	Payment handler, provider adapter, retry tests.
Decision	A logical payment operation must reuse a stable idempotency key.
Historical evidence	Prior timeout incident and associated regression test.
Constraints	Public checkout API must not change; no duplicate payment side effects.
Hypothesis	Retry path may generate a new key per attempt. Unverified until reproduced.

6.3 Memory write and trust policy

Classification	Rule
Verified fact	Supported by current code, trusted test output, or authoritative project state.
Approved decision	Accepted by an authorized human or policy; store version and rationale.
Hypothesis	Potential explanation that must remain explicitly unverified.
Historical observation	Bound to a specific run, commit, environment, and timestamp.
Obsolete/stale	No longer describes current state; retain history but avoid presenting it as current.

6.4 Freshness and invalidation

When code changes, mark derived summaries and indexes for affected files as stale and refresh them after integration. Do not delete historical records simply because they are old: an old decision may still explain a past change. Track source commit, ticket, run, timestamp, verification status, and artifact references for important records.

6.5 Recommended implementation

Use PostgreSQL for structured project and execution state; Git for source and version history; object storage for large reports and patches; PostgreSQL full-text search and pgvector for retrieval when useful. Add a dedicated graph database only if measured graph queries justify the operational cost.

SECTION 07

Data Model and Persistence

7.1 Core entities

Entity	Key fields / notes
users	id, email, created_at
projects	id, owner_id, name, status, settings
memberships	project_id, user_id, role
repositories	project_id, provider, repository_url, default_branch
tickets	id, project_id, title, description, type, priority, status
tasks	id, ticket_id, parent_task_id, role, status, attempt_count
task_dependencies	task_id, depends_on_task_id; enforce no cycles at workflow level
agent_runs	id, task_id, model, status, base_commit, start/end timestamps
tool_calls	id, run_id, tool_name, status, input_ref, output_ref
artifacts	id, run_id, type, storage_key, content_hash
task_events	id, task_id, event_type, payload, created_at
memory_records	id, project_id, type, content, source_ref, version, validity_status
pull_requests	id, ticket_id, repository_id, external_id, status
check_results	id, pull_request_id, check_name, status, artifact_ref
approvals	id, target_type, target_id, actor_id, decision, timestamp
usage_records	id, run_id, model, input/output tokens, estimated cost

7.2 Persistence rules

- Use foreign keys, uniqueness constraints, transactions, and tenant-scoped authorization.
- Use stable identifiers and schema versions for long-lived events and artifacts.
- Index common queries such as (project_id, status), (ticket_id, status), and (task_id, created_at).
- Keep large logs, patches, and test reports in object storage when appropriate.
- Store important state transitions in an append-only event history.
- Use idempotency keys for job dispatch and external actions that may be retried.
- Avoid putting secrets, full repository contents, or unbounded logs directly into model prompts or ordinary database rows.

7.3 Data relationships

A project has repositories, tickets, memberships, and memory records. A ticket decomposes into tasks. Tasks have dependencies and may have multiple agent runs. Runs create tool calls and artifacts. Tickets may produce pull requests, which have check results and approvals. All relevant records link back to a project and specific code revision for auditability.

SECTION 08

API and Event Design

Use a versioned REST API for commands and queries. Use Server-Sent Events (SSE) for one-way live progress from server to browser; WebSockets are not required unless bidirectional real-time collaboration becomes a real need.

Endpoint	Purpose
`POST /api/v1/projects`	Create a project from an idea
`GET /api/v1/projects/:id`	Get project state and configuration
`POST /api/v1/projects/:id/build-runs`	Start initial application build
`POST /api/v1/projects/:id/tickets`	Create a bug or feature ticket
`GET /api/v1/tickets/:id`	Get ticket details and lifecycle status
`POST /api/v1/tickets/:id/runs`	Start ticket execution workflow
`GET /api/v1/runs/:id`	Get run state and summary
`POST /api/v1/runs/:id/cancel`	Request safe cancellation
`GET /api/v1/runs/:id/events`	Stream authorized run events
`GET /api/v1/projects/:id/events`	Stream project events
`GET /api/v1/projects/:id/memory/search`	Search project knowledge
`GET /api/v1/runs/:id/artifacts`	List run artifacts
`POST /api/v1/pull-requests/:id/approvals`	Record authorized approval

8.1 Example event

```
{
  "event_id": "evt_901",
  "run_id": "run_456",
  "sequence": 18,
  "type": "task.verification_failed",
  "timestamp": "2026-10-27T04:30:00Z",
  "data": {
    "task_id": "task_789",
    "check": "payment.integration.test",
    "artifact_id": "artifact_321"
  }
}
```

8.2 Event delivery and reconnect

Persist event history and expose a sequence number or cursor. The UI can reconnect from the last seen event without losing progress. Use a transactional outbox or equivalent mechanism so a database state change and queue publication do not silently diverge.

8.3 API standards

- Validate request bodies and agent outputs against versioned schemas.
- Use consistent error formats, pagination, request IDs, and rate limits.
- Authorize every resource access; never rely only on UI hiding controls.
- Long-running work returns a run ID quickly; it must not occupy a request thread for the full build.
- Record actor, action, target, decision, and timestamp for security-sensitive operations.

SECTION 09

Orchestration, Reliability, and Concurrency

9.1 Scheduler responsibilities

- Validate the task graph and reject cycles.
- Find tasks whose dependencies have completed successfully.
- Check project-level and global concurrency limits.
- Enforce per-run budgets, deadlines, and allowed tool permissions.
- Dispatch eligible tasks and persist state transitions.
- Handle completion, failure, timeout, cancellation, and worker loss.
- Resume from durable checkpoints and preserve artifacts from previous attempts.

9.2 Delivery semantics

> **AT-LEAST-ONCE, NOT EXACTLY-ONCE**
> Assume queue jobs may be delivered more than once. Use stable job IDs, idempotency keys, conditional state transitions, and deduplication for side effects. Do not promise exactly-once execution across external systems; design for at-least-once delivery and safe, idempotent processing.

9.3 Failure scenarios

Failure	Expected behavior
Worker crashes	Lease expires or heartbeat is lost; task becomes retryable and resumes from a safe checkpoint.
Model timeout	Record failure, apply bounded retry/backoff, and respect the overall run deadline.
Duplicate queue delivery	Detect existing task/run state and avoid repeating completed side effects.
Tests repeatedly fail	Stop after retry budget and escalate with evidence.
Agent output invalid	Reject schema, ask for a bounded correction, or mark the task failed.
Conflicting patches	Detect overlap; serialize, rebase, or create an explicit conflict-resolution task.
Budget exhausted	Pause safely and request approval or a budget change.
User cancels	Stop scheduling new work, signal active workers, and clean up safely.
Memory source stale	Mark it stale, refresh or revalidate, and avoid treating it as current fact.

9.4 Parallel execution

Parallelize independent tasks after their interfaces are stable. Each coding task works in an isolated Git worktree or equivalent workspace. Before integration, check the base commit, changed files, patch compatibility, tests, and merge conflicts. A dedicated integration step is responsible for combining work and rerunning affected checks.

SECTION 10

Security and Trust Boundaries

AgentForge accepts untrusted text, repository files, dependency metadata, tool output, and generated code. Security must be enforced by deterministic services and worker isolation, not only by prompts or a Security Agent.

Threat	Mitigation
Prompt injection in tickets/repository	Treat retrieved content as untrusted data; tools enforce permissions outside the model.
Arbitrary code execution	Run generated code in isolated, resource-limited workers with restricted networking.
Cross-project leakage	Tenant-scoped authorization and access checks for every resource and retrieval query.
Secret exposure	Short-lived credentials, secret redaction, scoped tokens, restricted environment variables.
Dangerous tool call	Allowlisted tools, argument validation, least privilege, audit logs.
Infinite loops or runaway costs	Max steps, wall-clock deadlines, token/tool budgets, bounded retries.
Malicious dependencies	Restricted egress, dependency policy, disposable build environments.
Unauthorized release	Separate release permissions and explicit approval gates.
Forged verification claims	Accept results only from trusted worker records bound to a code revision.
Memory poisoning	Track provenance; separate hypothesis from verified fact; version and revalidate knowledge.

10.1 Trust boundaries

- **Browser → API:** authenticate and authorize every request.
- **API → orchestrator:** validate state transitions and budgets.
- **Agent → tools:** enforce allowlists, schemas, and scoped permissions.
- **Worker → repository:** isolate workspaces and limit credential access.
- **Worker → external network:** deny by default or allow only explicitly approved destinations.
- **Verification → release:** only trusted, revision-bound check results satisfy gates.

10.2 Human approval gates

AUTHORIZATION REQUIRED

Require explicit authorization for merging, deploying, changing production data, granting new permissions, or performing other consequential actions. Agents can prepare a proposal and supporting evidence; the policy engine and authorized human decide whether it proceeds.

SECTION 11

Observability, Cost, and Evaluation

11.1 Observability model

- Correlate project, ticket, task, run, tool call, artifact, commit, and verification IDs.
- Record structured logs, state transitions, durations, retries, errors, and tool outcomes.
- Track model, input/output tokens, estimated cost, latency, and budget consumption per run.
- Capture the exact code revision used for tests and review.
- Expose a timeline in the UI so a developer can reconstruct why the system acted.

11.2 Evaluation metrics

Metric	Definition / purpose
Verified ticket completion rate	Share of benchmark tickets meeting predeclared acceptance checks.
Test pass rate	Share of generated changes passing required test suites.
Regression rate	Frequency of changes that break existing behavior.
Repair iterations	Number of implementation/verification loops per ticket.
Time to verified completion	Elapsed time from ticket acceptance to verified candidate.
Cost per ticket	Actual/estimated model and execution cost for a ticket.
Worker recovery rate	Share of injected worker failures that recover without losing durable state.
Human rejection rate	Share of proposed changes rejected by reviewers.
Tool failure rate	Share of tool calls failing or being rejected by policy.

11.3 Benchmark strategy

Create a small, repeatable set of tickets against a controlled repository: a simple feature, a reproducible bug, a regression, a security-sensitive change, and a task with independent subtasks. Record baseline performance for a single-agent workflow and compare it with the multi-agent workflow. Do not claim improvements until repeated runs support them.

11.4 Quality gates

- Acceptance criteria must be machine-checkable where practical.
- Test output must come from a trusted worker and identify the code revision.
- Security findings require evidence and a defined severity policy.
- Release readiness is based on check results and approvals, not model confidence.

SECTION 12

Frontend and Deployment Design

12.1 Primary frontend screens

Screen	Key content
Project overview	Build status, repository, ticket backlog, pull requests, release status, usage.
Ticket workspace	Description, acceptance criteria, task graph, related memory, artifacts, current state.
Live run view	Active agents, queued tasks, tool calls, failures, repair loops, costs and elapsed time.
Code review	Changed files, diff, test evidence, security findings, comments, approval actions.
Project Brain	Searchable decisions, prior incidents, code links, provenance, freshness indicators.
Settings and policy	Repository access, agent permissions, budgets, approved tools, release policy.

12.2 Deployment topology

Web frontend and API are deployed separately from long-running workers. PostgreSQL stores authoritative state, Redis supports queueing, and object storage holds large artifacts. Workers receive short-lived scoped credentials and run in isolated environments. CI is integrated through the repository provider. OpenTelemetry-compatible instrumentation captures service and run traces.

12.3 Local development

- Docker Compose for PostgreSQL, Redis, API, frontend, and worker.
- Seed repository and deterministic benchmark tickets.
- Mock model adapter for fast tests and repeatable failure scenarios.
- Real model provider enabled through environment configuration and budget limits.
- No production credentials or unrestricted deployment access in local sandboxes.

12.4 Operational backups

Back up PostgreSQL, protect object storage, retain Git history, and test restoration. Treat workflow state and artifacts as essential operational data: a restored system should be able to explain what happened and resume or safely terminate interrupted runs.

SECTION 13

Seven-Day Implementation Plan

This is an intensive capstone for learners who already understand basic programming. The goal is a working vertical slice, not a universally capable autonomous company. Provide a starter repository or reduce scope for beginner cohorts.

Day	Focus	Deliverable
1	System design	Requirements, architecture diagram, ERD, task DAG, memory design, threat model, API contracts, implementation plan.
2	Foundation	Authentication, projects, tickets, migrations, repository connection, model adapter.
3	Orchestration	Task scheduler, agent runtime, structured tools, persisted state, event streaming.
4	Initial build	Repository tools, isolated workspace, code generation, real test execution, artifact capture.
5	Memory and ticket repair	Context builder, code index, retrieval, bug investigation, bounded repair loop.
6	Security and reliability	Permissions, worker isolation, retries, recovery, observability, evaluation harness.
7	Integration and demo	Deployment, end-to-end benchmark, failure recovery demo, docs, measured results.

Day 1 workshop agenda (about five hours)

Session	Time	Output
Product scope and user journeys	45 min	Requirements and acceptance criteria
Multi-agent architecture	60 min	Component diagram and boundaries
Workflow and coordination	60 min	State machine and task DAG
Data model and APIs	60 min	ERD, API contracts, event schemas
Reliability and security	45 min	Failure analysis and threat model
AI design critique	30 min	Revised architecture and trade-offs

Day 1 optional proof of concept

If time permits, run a tiny experiment: one orchestrator invokes two specialized agents, validates their structured outputs, and records the run. The goal is to test the riskiest assumption, not to begin full implementation before the design is understood.

SECTION 14

Scope, Risks, and Final Demonstration

14.1 Must ship versus stretch goals

Must ship	Advanced / stretch
One supported stack and repository workflow	Multiple languages and repository providers
Project/ticket lifecycle and persisted state	Complex portfolio-wide planning
One or two coding roles plus QA/review	Dynamic agent hiring and role evolution
Isolated execution and real tests	Multiple sandbox backends and advanced isolation
Basic Project Brain and exact retrieval	Graph-based knowledge traversal and advanced reranking
Bounded repair loop and human approval	Autonomous multi-stage release automation
Run tracing and basic cost metrics	Large-scale evaluation and adaptive model routing

14.2 Main project risks

- **Scope explosion:** limit languages, integrations, agent roles, and release targets.
- **Unreliable generated code:** use small tasks, real tests, code review, and a bounded repair loop.
- **Memory poisoning:** record provenance and distinguish verified facts from hypotheses.
- **Concurrency conflicts:** isolate workspaces and validate integration.
- **Cost runaway:** impose budgets, stop conditions, and project-level limits.
- **False completion:** bind verification to a specific code revision and trusted worker result.
- **Unsafe execution:** isolate generated code and keep production credentials out of workers.

14.3 Final demo script

1. Submit an idea for a small application.
2. Show requirements and architecture artifacts.
3. Show the generated task DAG and agent assignments.
4. Build a small working slice in a Git repository.
5. Submit a bug ticket against that application.
6. Show Project Brain retrieving a related decision or prior run.
7. Have an agent reproduce and fix the issue.
8. Run actual tests and show a reviewable patch or pull request.
9. Inject a worker failure in a separate run and demonstrate recovery.
10. Show traceability, memory provenance, token cost, and benchmark results.

14.4 Definition of done

- The initial idea becomes a structured plan and a small working application.
- A bug ticket can be processed against the existing repository.
- Relevant memory is retrieved and linked to its source.
- The patch is executed and verified in a controlled environment.
- Failed checks lead to bounded repair or safe escalation.
- The final change is reviewable and requires authorization before merge/deploy.
- A benchmark records quality, time, cost, and failure recovery.

SECTION 15

Architecture Decisions and Glossary

15.1 Recommended architecture decisions

Decision	Recommendation	Reason
Application shape	Modular monolith + isolated workers	Simpler initial implementation with a strong code-execution boundary.
Workflow ownership	Durable state machine outside agents	Predictable retries, recovery, cancellation, and auditability.
Agent model	Role-configured workers with contracts	Roles can share a model while retaining bounded tools and permissions.
Memory authority	Git + structured DB + verified artifacts	Retrieval helps find knowledge but does not override authoritative state.
Search	Exact search first; semantic search where useful	Avoid unnecessary complexity and improve provenance.
Live UI updates	SSE plus durable event history	Sufficient for server-to-browser progress and reconnect.
Code isolation	Separate restricted execution workers	Generated code is untrusted and must not share API privileges.
Release policy	Human approval for merge/deploy	Keep consequential actions explicitly authorized.

15.2 Glossary

Term	Meaning
Agent runtime	Environment that invokes a model with role instructions, tools, context, and budgets.
Orchestrator	Deterministic controller that schedules work and owns workflow state.
Task DAG	Directed acyclic graph representing tasks and their dependencies.
Agent run	A single execution attempt for a task.
Artifact	A durable output such as a patch, test report, log, or architecture document.
Project Brain	Memory and retrieval subsystem for project-specific knowledge.
ADR	Architecture Decision Record explaining a decision and its rationale.
Idempotency	Repeated execution of an operation does not create duplicate effects.
Context packet	Task-specific, permission-checked information assembled for an agent.
Provenance	Record of where information came from and how it was verified.
Repair loop	Bounded cycle of failed verification, diagnosis, patch, and re-test.

FINAL DESIGN PRINCIPLE

AgentForge combines three forms of intelligence: **agent intelligence** (planning, coding, debugging), **system intelligence** (orchestration, permissions, retries, verification), and **project intelligence** (persistent knowledge of the product and its history). A reliable system needs all three.

The goal is not to claim that a week produces a fully autonomous software company. The goal is to build a credible, demonstrable vertical slice that creates an application, resolves a ticket against its existing code, verifies the result, remembers what happened, and explains every important action.